EIM Platform Business Proposal

https://gemini.google.com/app/c1e9e399b32ada19

User prompt: I am planning to build an enterprise incentive management platform, I am the product manager of this new proposition, help me write a business proposal for this EIM. Enterprise incentive management (EIM) or incentive compensation management (ICM) software automates the design, calculation, tracking, and auditing of complex sales commissions, bonuses, and incentive programs at scale. [1, 2] Top Enterprise Incentive Management Platforms - CaptivateIQ: Best for mid-market and enterprise teams needing a flexible ICM with spreadsheet-style no-code workbooks, AI-crafted earnings explanations, and full sales performance management (SPM) capabilities. Explore the platform on the CaptivateIQ Website. [1] - Xactly (Xactly Incent): Best for large enterprises requiring mature commission automation, built-in industry pay and performance benchmarking, and automated ASC 606 / IFRS 15 commission accounting. Learn more via the CaptivateIQ ICM Guide. [1] - Varicent: An AI-native incentive compensation and sales performance management platform ranked highly for automated workflows, predictive plan modeling, and data orchestration. Review capabilities on the Varicent Products Page. [1] - SAP Incentive Compensation Management: A cloud-deployed solution designed to model sales plans via drag-and-drop templates, cut payment errors, and drive behavior via prescriptive AI analytics. Visit the SAP Incentive Compensation Management Portal. [1] - Salesforce Spiff: Offers customizable rep commission statements, real-time tracking, an integrated commission estimator, and annual pricing around $75 USD/user/month. Check features on the Salesforce AP Product Page. [1] - Other Platforms: Additional enterprise-grade SPM and incentive options include Akeron (Vulki), Anaplan, beqom, and Forma.ai, alongside channel incentive platforms like ITA Group and BI WORLDWIDE. Compare additional options on the Anywherer ICM Review. [1, 2] Key Enterprise Capabilities - No-Code Plan Modeling: Allows compensation admins to build or adjust commission structures without heavy IT intervention. [1, 2] - Automated Data Sync: Integrates directly with CRM (e.g., Salesforce), ERP, HCM, and payroll systems to pull accurate deal and credit data. [1] - Audit Trails & Compliance: Locks historical statements and logs every plan change or manual override to satisfy financial governance (ASC 606 / IFRS 15). [1, 2] - Seller Transparency: Gives sales reps real-time dashboards to track quota progress, estimate future commissions, and submit disputes. [1, 2] If you'd like to narrow down the right platform, tell me: - Are you focusing on direct sales commissions or partner/channel incentives? - How many users/sellers are on the plan? - Which CRM or payroll systems do you currently use?

Response: Here is a comprehensive, executive-ready **Business Proposal (Product Strategy & Investment Case)** tailored for your new **Enterprise Incentive Management (EIM)** proposition. It leverages the competitive insights and core capabilities you outlined while framing a clear market wedge to justify building a next-generation platform.

Business Proposal: Next-Generation Enterprise Incentive Management (EIM) Platform

Prepared by: Product Management

Subject: Investment Case & Product Strategy for an AI-Native, Composable EIM Solution

Target Market: Mid-Market & Enterprise Organizations (250–5,000+ Commissionable Headcount)

1. Executive Summary

Enterprise revenue organizations spend billions annually on variable compensation, yet the administration of these incentives remains caught between two flawed extremes: **error-prone spreadsheets** that lack audit controls, or **rigid legacy EIM platforms** that require months of costly IT implementation and consulting support to adapt to changing go-to-market (GTM) models.

We propose building a modern, **AI-native Enterprise Incentive Management (EIM)** platform that combines the agility of no-code, spreadsheet-style modeling with enterprise-grade governance (ASC 606 / IFRS 15) and real-time seller behavioral motivation. By bridging the gap between Finance, RevOps, and Sales, our platform will reduce commission processing time by up to 80%, eliminate payout leakage, and turn compensation plans into active revenue drivers rather than static post-mortem statements.

2. Problem Statement & Market Opportunity

The Core Friction Points

1. **The Agility vs. Governance Trade-off:** Legacy enterprise systems (e.g., SAP ICM, Xactly, Varicent) offer robust compliance and audit trails, but modifying a compensation plan mid-year—such as adding usage-based pricing spiffs or product overlays—requires expensive technical resources. Conversely, manual spreadsheets break at scale and fail financial audits.
2. **Data Fragmentation for Modern Revenue Models:** B2B enterprises are shifting from simple SaaS seat-licensing to hybrid models (subscriptions + consumption/usage + partner-led channel sales). Existing ICM tools struggle to ingest high-volume metering data alongside traditional CRM/ERP data in real time.
3. **"Shadow Accounting" by Sellers:** Because reps do not understand complex multi-tier accelerators or clawback rules, they spend hours every month maintaining personal spreadsheets to verify their payouts—eroding trust and taking time away from selling.

Market Wedge (Why Build Now?)

While incumbents like **CaptivateIQ** and **Salesforce Spiff** have modernized the user interface, a massive white space remains for an **AI-First, Composable EIM** that:

- Directly reads cloud data warehouses (Snowflake, BigQuery, Databricks) without fragile ETL pipelines.
- Uses Generative and Predictive AI not just to explain statements, but to **simulate plan ROI** for Finance before rollout.
- Natively unifies **Direct Sales Commissions** and **Partner/Channel Rebates & Incentives** in a single calculation engine.

3. Competitive Landscape & Strategic Positioning

To win against established leaders, our product must be positioned around speed-to-value, calculation flexibility, and intelligent optimization:

Competitor Segment	Key Players	Their Strengths	Their Vulnerabilities	Our Differentiation ("The Wedge")
Legacy Enterprise	Xactly, SAP ICM, Varicent	Deep ASC 606 accounting, benchmarking data, massive scale	Long implementation (6–12 mos), rigid schemas, heavy professional services reliance	**10x faster deployment** via zero-copy data integration and intuitive no-code logic builders.
Modern Agile ICM	CaptivateIQ, Salesforce Spiff	Spreadsheet-like UI (CaptivateIQ), tight CRM coupling & ~$75/user/mo benchmark (Spiff)	Performance bottlenecks at massive data volumes; Spiff is tightly bound to the Salesforce ecosystem	**CRM-agnostic architecture** built on high-throughput engines capable of handling usage/consumption billing at scale.
Broad Planning / Niche	Anaplan, beqom, Forma.ai	Enterprise-wide financial modeling or managed-service flexibility	High TCO, steep learning curve, or lack of self-serve RevOps autonomy	**Purpose-built self-serve RevOps UX** paired with automated AI plan diagnostics.

4. Proposed Product Architecture & Core Capabilities

Our platform will be structured around four foundational pillars designed to serve **RevOps/Comp Admins**, **Finance/Controllers**, and **Sellers/Partners**:

Pillar 1: Composable, No-Code Calculation Engine (For RevOps)

- **Familiar Grid + Natural Language Modeling:** Allow admins to build complex multi-tier hurdles, draws, caps, and team splits using either spreadsheet-syntax formulas or natural-language AI prompts (*"Create a 1.5x accelerator on net-new ARR above 100% quota with a 60-day clawback"*).
- **Effective-Dated Versioning:** Seamlessly handle mid-quarter territory realignments, role promotions, and leave-of-absence prorations without breaking historical calculations.

Pillar 2: Zero-Copy Automated Data Sync (For IT & Data Teams)

- **Turnkey Connectors:** Out-of-the-box bi-directional sync with major CRMs (Salesforce, HubSpot, Microsoft Dynamics), ERPs (NetSuite, SAP), and HCM/Payroll systems (Workday, ADP, Deel).
- **Warehouse-Native Ingestion:** Direct connection to data warehouses to calculate commissions on product-led growth (PLG) and consumption/metering events alongside standard CRM closed-won deals.

Pillar 3: Financial Governance, Audit & Compliance (For Finance)

- **Automated ASC 606 / IFRS 15 Accounting:** Automatically calculate cost-of-obtaining-a-contract capitalization, amortization schedules, and journal entries ready for ERP export.
- **Immutable Audit Trails & Approval Workflows:** Lock historical payout periods, track every manual override or credit split adjustment with user timestamps, and enforce multi-stage sign-offs before payroll disbursement.
- **AI Sandbox &Monte Carlo Plan Modeling:** Allow Finance to stress-test next year's compensation plans against historical deal data to forecast total payout liability and identify edge-case windfalls before launch.

Pillar 4: Seller Transparency & Behavioral Motivation (For Sales & Channel)

- **Real-Time Earnings & Quota Dashboards:** Embedded directly within the seller's CRM workflow, showing real-time attainment, pending pipeline commissions, and pace-to-target.
- **Interactive "What-If" Commission Estimator:** Enable reps to select open opportunities in their pipeline and immediately see how adding a multi-year term or specific product bundle impacts their take-home pay.
- **GenAI Earnings Explainer & Dispute Resolution:** Provide an AI assistant on every commission statement that explains *why* a deal paid out a specific amount, drastically reducing manual dispute tickets for RevOps.

5. Business Model & Pricing Strategy

To remain competitive with market benchmarks (such as Salesforce Spiff's ~$75 USD/user/month anchor) while capturing enterprise upside, we recommend a **Tiered SaaS Subscription Model**:

1. **Platform Base Fee:** Annual foundational fee covering core integrations, sandbox environments, and security/compliance infrastructure ($15,000 – $40,000/year depending on enterprise tier).
2. **Per-Payee Seat Licensing:**
    - **Core Tier ($55 – $65 / active payee / month):** Automated data sync, no-code plan modeling, rep statements, dispute workflows, and standard CRM integrations.
    - **Enterprise AI & Compliance Tier ($80 – $95 / active payee / month):** Full ASC 606 / IFRS 15 accounting automation, AI predictive plan modeling, unlimited historical audit retention, and high-volume consumption billing support.
    - **Channel / Partner Add-On:** Volume-based pricing per active partner portal or transaction tier for external rebate/spiff management.

6. Phased Product Roadmap & Go-To-Market (GTM)

Phase 1: MVP & Design Partner Validation (Months 1–6)

- **Scope:** Core no-code calculation engine, native Salesforce + HubSpot + CSV/Snowflake ingestion, customizable rep statements, dispute ticketing, and payroll export.
- **Target:** 5–10 mid-market design partners (100–300 reps) migrating off spreadsheets or legacy tools.
- **Success Metric:** 100% parallel-run calculation accuracy against legacy payouts across two consecutive pay cycles.

Phase 2: Enterprise Governance & AI Layer (Months 7–12)

- **Scope:** ASC 606 / IFRS 15 amortization engine, multi-currency support, RBAC/SSO, GenAI statement explainer, and embedded CRM "What-If" deal calculator.
- **Target:** General Availability (GA) launch targeting Upper Mid-Market and Enterprise accounts (300–1,500+ reps).
- **Success Metric:** Reduce customer commission close cycle from 10+ business days to less than 2 days.

Phase 3: Full SPM & Channel Expansion (Months 13–18)

- **Scope:** Predictive Monte Carlo compensation modeling, Territory & Quota Planning (T&Q) modules, and external Partner/Channel incentive portals.
- **Target:** Full replacement of legacy SPM suites (Varicent, Xactly, Anaplan) in large enterprises.

7. Key Business Case KPIs (ROI for Buyers)

To win budget approval from CFOs and CROs, our sales collateral and product telemetry will prove ROI across four metrics:

- **Payout Accuracy:** Eliminate the industry-average **3%–8% commission overpayment leakage** caused by spreadsheet formula errors.
- **Admin Efficiency:** Cut monthly commission processing and reconciliation time by **70%–85%**.
- **Audit Readiness:** Reduce external audit preparation and ASC 606 compliance hours by **90%**.
- **Revenue Lift:** Drive a **5%–12% increase in target product attachment/multi-year deals** via real-time rep commission estimators during deal negotiation.

User prompt: Write a detailed Product Requirements Document (PRD) for Phase 1 MVP of this EIM platform, including user stories, data schema requirements, and acceptance criteria.

Response: Here is the detailed **Product Requirements Document (PRD)** for the **Phase 1 MVP** of your Enterprise Incentive Management (EIM) platform, designed for engineering, design, and executive alignment.

Product Requirements Document (PRD): EIM Platform – Phase 1 MVP

Document Metadata	Details
Product Name	Next-Gen Enterprise Incentive Management (EIM) – Core Engine & Seller Portal
Target Release	Phase 1 MVP (Months 1–6)
Target Customers	5–10 Mid-Market Design Partners (100–300 Commissionable Payees)
Primary Personas	Comp Admin (RevOps), Finance Approver (Controller), Payee (Sales Rep / Manager)

1. MVP Objective & Success Metrics

1.1 Objective

Deliver a reliable, auditable, no-code commission calculation engine and seller transparency portal that replaces manual spreadsheets and legacy ICM tools, enabling design partners to execute a complete monthly/quarterly pay cycle from CRM ingestion to payroll export.

1.2 MVP Release Exit Criteria (KPIs)

- **Parallel-Run Accuracy:** 100.0% calculation match down to the cent ($0.01) against design partners' existing shadow spreadsheets/legacy systems across two consecutive pay periods.
- **Calculation Performance:** Compute payouts for 500 payees across 100,000 transaction rows in less than 15 seconds.
- **Admin Autonomy:** 100% of standard SaaS commission plans (tiered accelerators, splits, draws, clawbacks) configured by RevOps without engineering intervention.
- **Close Cycle Reduction:** Reduce monthly commission reconciliation and approval workflows from 10+ business days to less than 3 business days.

2. Scope Boundary (In-Scope vs. Out-of-Scope)

Feature Area	In-Scope for Phase 1 MVP	Out-of-Scope (Deferred to Phase 2 / 3)
Data Ingestion	Salesforce native connector, HubSpot native connector, Snowflake read-only sync, CSV/XLSX manual upload with schema validation.	Bi-directional ERP sync (NetSuite/SAP), native Workday HCM write-back, real-time webhook streaming.
Plan Modeling	Spreadsheet-style formula grid, multi-tier rate tables, team/manager rollups, deal splits, recoverable/non-recoverable draws, clawbacks, effective dating.	Natural-language AI plan builder, Monte Carlo payout simulations, Territory & Quota (T&Q) planning.
Governance & Finance	Period locking, immutable calculation audit logs, manual adjustment overrides with mandatory reason codes, multi-step sign-off, CSV payroll export.	Automated ASC 606 / IFRS 15 capitalization & amortization journal entries.
Seller Experience	Web-based seller dashboard, downloadable PDF/CSV statements, line-item deal drill-down, dispute submission & tracking workflow.	GenAI earnings explainer chat, embedded CRM iframe widget, external Partner/Channel rebate portal.

3. Core Epics, User Stories & Acceptance Criteria

Epic 1: Data Integration & Credit Assignment Engine

Goal: Ingest raw transactional and organizational data cleanly and map transactions to the right payees.

- **US 1.1 (CRM & Warehouse Sync):** *As a Comp Admin, I want to map objects from Salesforce/HubSpot (Opportunities, Line Items, Users) or Snowflake tables into the EIM data model on a scheduled or on-demand basis.*
    - **Acceptance Criteria:**
        - Admin can authenticate via OAuth 2.0 (CRM) or key-pair auth (Snowflake) in less than 3 minutes.
        - System validates incoming records against required schema types (e.g., `Close_Date` as Date, `ARR_Amount` as Decimal) and flags rejected rows in an error log without failing the entire sync batch.
        - Incremental sync pulls only modified records (`LastModifiedDate` greater than last sync timestamp) and preserves raw source snapshots for auditability.
- **US 1.2 (Credit Splits & Hierarchy Rollups):** *As a Comp Admin, I want to split deal credit across multiple reps (e.g., 70% AE1 / 30% AE2) and automatically roll up 100% of credit to their respective Sales Managers and Directors.*
    - **Acceptance Criteria:**
        - System supports percentage splits totaling less than, equal to, or greater than 100% (for double-crediting overlays like Sales Engineers).
        - Reporting hierarchies are **effective-dated**: if Rep A moves from Manager X to Manager Y on May 15, deals closed May 1–14 roll up to Manager X, and deals closed May 15–31 roll up to Manager Y automatically.

Epic 2: No-Code Plan & Rule Builder

Goal: Allow RevOps to build complex compensation logic using familiar spreadsheet syntax and reusable rate tables.

- **US 2.1 (Multi-Tier Attainment & Accelerators):** *As a Comp Admin, I want to build marginal and non-marginal rate tables so that deals crossing quota thresholds are paid accurately—including split-tier marginal math on the exact dollar that crosses a threshold.*
    - **Acceptance Criteria:**
        - Admin can toggle a rate table between **Marginal (Progressive)** and **Retroactive (Cliff)** calculation modes.
        - *Edge-Case Verification:* If a rep is at $90,000 attainment on a $100,000 quota (10% base rate, 15% accelerated rate above 100%) and closes a $20,000 deal, the engine must split the single transaction into $10,000 at 10% ($1,000) + $10,000 at 15% ($1,500) = $2,500 payout, and log both tier breakdowns on the statement.
- **US 2.2 (Draws, Caps, and Clawbacks):** *As a Comp Admin, I want to configure recoverable/non-recoverable draws for ramping reps and automated clawbacks when a customer churns within a defined window (e.g., 90 days).*
    - **Acceptance Criteria:**
        - Recoverable draw deficits automatically carry forward to subsequent open pay periods until recouped.
        - When an ingested transaction is flagged as `Cancelled/Churned` within the plan's clawback window, the engine generates a negative offsetting ledger entry in the **current open period** while keeping the historical closed period untouched.

Epic 3: Financial Governance, Period Close & Audit Trail

Goal: Provide Finance Controllers with airtight controls, change logs, and period-locking mechanisms.

- **US 3.1 (Manual Overrides & Audit Logging):** *As a Comp Admin or Finance Approver, I want to apply manual adjustments (e.g., spot bonuses or dispute settlements) to a statement while enforcing a mandatory comment and preserving the original calculated value.*

    - **Acceptance Criteria:**
        - Every manual override writes an immutable record to the `Audit_Log` table containing: `Actor_ID`, `Timestamp`, `Entity_ID`, `Previous_Value`, `New_Value`, and `Reason_Code`.
        - No user (including System Admins) can delete or mutate audit log entries.
- **US 3.2 (Hard Period Lock & Payroll Export):** *As a Finance Approver, I want to freeze a payout period after review so that subsequent CRM changes cannot alter historical commission statements, and export a payroll-ready file.*
    - **Acceptance Criteria:**
        - Transitioning a `Payout_Period` status to `LOCKED` snapshots all underlying calculation inputs and blocks recalculation.
        - Retroactive CRM changes affecting a locked period are automatically routed to a **True-Up Queue** to be applied as prior-period adjustments in the current open period.
        - Payroll export generates customizable CSV templates formatted for ADP, Workday, and NetSuite journal summaries.

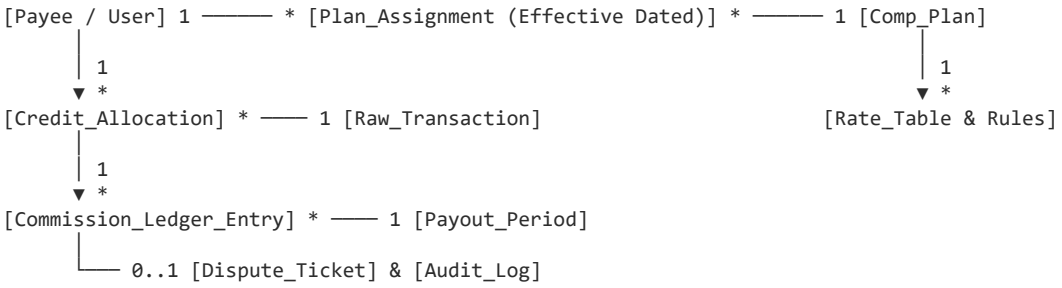
Epic 4: Seller Transparency & Dispute Workflow

Goal: Eliminate shadow accounting by giving reps clear visibility into their earnings and a structured way to raise questions.

- **US 4.1 (Interactive Commission Statement):** *As a Sales Rep, I want to view my current period earnings, quota attainment progress bar, and a deal-by-deal breakdown showing how each commission amount was calculated.*
    - **Acceptance Criteria:**
        - Rep can filter transactions by payout period, deal name, and product type.
        - Clicking any deal row opens a **Calculation Trace drawer** displaying: Deal Amount → Split % → Credited Amount → Quota Attainment Tier Applied → Commission Rate → Final Payout.
- **US 4.2 (In-App Dispute Resolution):** *As a Sales Rep, I want to flag a specific transaction or missing deal directly from my statement and track its resolution status.*
    - **Acceptance Criteria:**
        - Rep can submit a dispute attached to a specific `Transaction_ID` (or general period) with notes and file attachments.
        - Comp Admin receives a queue notification, can approve/reject the dispute with comments, and if approved, trigger a one-click adjustment that links directly to the dispute ticket ID.

4. Core Data Schema Requirements

To support effective-dated hierarchies, immutable audits, and split-tier calculations, the relational database (PostgreSQL) combined with the analytical calculation store must implement the following core entities:



Key Table Specifications

1. `Payee` (Users & Hierarchy)

- `payee_id` (UUID, PK)
- `external_crm_id` (VARCHAR, Indexed)
- `employee_number` (VARCHAR, Unique – for Payroll mapping)
- `full_name`, `email`, `role_title`, `base_currency` (ISO 4217)
- `manager_payee_id` (UUID, FK self-reference)
- `effective_start_date` (DATE, NOT NULL)
- `effective_end_date` (DATE, Nullable – supports SCD Type 2 historical role/manager tracking)

2. `Plan_Assignment`

- `assignment_id` (UUID, PK)
- `payee_id` (UUID, FK)

- `plan_id` (UUID, FK)
- `quota_amount` (NUMERIC(18,4))
- `target_incentive_amount` (NUMERIC(18,4) – Variable OTE)
- `proration_factor` (NUMERIC(5,4) – Default 1.0000)
- `effective_start_date` (DATE), `effective_end_date` (DATE)

3. `Raw_Transaction` **(Ingested Deals / Invoices / Usage)**

- `transaction_id` (UUID, PK)
- `source_system` (ENUM: `SALESFORCE`, `HUBSPOT`, `SNOWFLAKE`, `CSV`)
- `external_deal_id` (VARCHAR, Indexed)
- `transaction_date` (DATE, Indexed – e.g., Close Date or Invoice Date)
- `booking_amount` (NUMERIC(18,4))
- `currency_code` (CHAR(3))
- `fx_rate_applied` (NUMERIC(12,6))
- `custom_attributes` (JSONB – stores dynamic fields like `Product_Family`, `Contract_Term_Months`, `Deal_Type` without schema migrations)
- `ingestion_batch_id` (UUID, FK)

4. `Commission_Ledger_Entry` **(The Immutable Output Grain)**

- `ledger_id` (UUID, PK)
- `payout_period_id` (UUID, FK)
- `payee_id` (UUID, FK)
- `transaction_id` (UUID, FK, Nullable for non-deal bonuses/draws)
- `plan_component_id` (UUID, FK – e.g., Base ARR Commission vs. Multi-Year Spiff)
- `credited_amount` (NUMERIC(18,4))
- `tier_applied_id` (UUID, FK – tracks exact marginal tier slice)
- `rate_applied` (NUMERIC(8,6))
- `payout_amount` (NUMERIC(18,4))
- `entry_type` (ENUM: `CALCULATED`, `MANUAL_OVERRIDE`, `DRAW_ADVANCE`, `DRAW_RECOVERY`, `CLAWBACK`, `PRIOR_PERIOD_TRUEUP`)
- `is_locked` (BOOLEAN, Default FALSE)

5. `Audit_Log` **(Append-Only System Table)**

- `audit_id` (UUID, PK)
- `timestamp_utc` (TIMESTAMPTZ, NOT NULL)
- `actor_user_id` (UUID, FK)
- `action_type` (ENUM: `PLAN_EDIT`, `MANUAL_ADJUSTMENT`, `PERIOD_LOCK`, `DISPUTE_RESOLUTION`, `DATA_OVERRIDE`)
- `target_entity` (VARCHAR), `target_record_id` (UUID)
- `before_state` (JSONB), `after_state` (JSONB)
- `reason_comment` (TEXT, NOT NULL for adjustments)

5. Non-Functional Requirements (NFRs)

1. **Numeric Precision & Determinism:**
   - All monetary and rate calculations must use arbitrary-precision decimal arithmetic (`NUMERIC(18,4)` minimum in storage, 8+ decimal places in intermediate calculation memory) with configurable rounding rules (Half-Even / Banker's Rounding vs. Half-Up) at the final payout step. **Floating-point (`FLOAT`/`DOUBLE`) types are strictly prohibited** to prevent sub-cent rounding drift.
2. **Security & Compliance (Enterprise Readiness):**
   - **SOC 2 Type II Architecture:** Encryption at rest (AES-256) and in transit (TLS 1.3).

- **Row-Level Security (RLS) & RBAC:** Payees can strictly query `Commission_Ledger_Entry` rows where `payee_id = current_user` (or downline hierarchy for Managers). Comp Admins and Finance Approvers have separated duties (e.g., Comp Admin can configure plans; only Finance Approver role can execute `PERIOD_LOCK`).
- **SSO Integration:** SAML 2.0 / OIDC support (Okta, Azure AD / Entra ID, Google Workspace) included in MVP.

3. **DAG Calculation Engine Architecture:**
   - Formulas and dependencies must compile into a **Directed Acyclic Graph (DAG)** to detect circular references at plan-save time and execute independent payee calculations in parallel worker threads.

6. MVP Rollout & Validation Plan (Months 1–6)

- **Months 1–2 (Foundation & Ingestion):** Deliver PostgreSQL core schema, RBAC/SSO, Salesforce/CSV connectors, and effective-dated hierarchy management.
- **Months 3–4 (Calculation DAG & No-Code Grid):** Deliver the formula parser, marginal rate-table engine, draw/clawback logic, and background worker execution.
- **Month 5 (Governance & Seller Portal):** Deliver the Rep Statement UI, Calculation Trace drawer, Dispute workflow, Period Lock, and Audit Log viewer.
- **Month 6 (Shadow Payroll & Design Partner Sign-Off):** Run 2 historical pay periods + 1 live pay period in parallel with 5 design partners to validate 100% cent-for-cent accuracy before turning off their legacy spreadsheets.

User prompt: Write a technical product specification for the Phase 2 ASC 606 / IFRS 15 commission accounting and amortization module.

Response: Technical Product Specification: ASC 606 / IFRS 15 Commission Accounting & Amortization Module (Phase 2)

Document Metadata	Details
Module Name	Incremental Costs of Obtaining a Contract (ASC 340-40 / ASC 606 & IFRS 15)
Target Release	Phase 2 (Months 7–12)
Primary Users	Revenue Accountants, SEC/Financial Reporting Managers, Controllers, External Auditors
Core Dependency	Phase 1 Immutable `Commission_Ledger_Entry` & Locked `Payout_Period`

1. Module Overview & Accounting Scope

1.1 Accounting Objective

Under **ASC 606 (specifically Subtopic 340-40)** and **IFRS 15**, incremental costs of obtaining a contract with a customer (such as sales commissions, manager overrides, and fringe benefits) must be **capitalized as a deferred contract cost asset** on the balance sheet and **amortized systematically** on a basis consistent with the transfer of goods or services to which the asset relates, rather than expensed immediately upon payout.

1.2 Core Capabilities Delivered

1. **Capitalization Rules Engine:** Automatically classifies every line item in `Commission_Ledger_Entry` as **Capitalizable** (incremental to winning a specific contract) or **Immediate Expense** (discretionary bonuses, non-incremental draws, or practical expedient contracts ≤ 12 months).
2. **Dual-Method Amortization Engine:** Supports both **Individual Contract-Level** tracking and **Portfolio-Level (Cohort)** tracking across Customer Life (Expected Benefit Period) or Initial Contract Term.
3. **Lifecycle Event & Impairment Processor:** Dynamically recalculates deferred asset balances and catch-up/impairment entries upon contract churn, downgrades, early renewals, or prior-period commission true-ups.
4. **Automated Subledger & ERP Journal Generator:** Produces balanced, multi-currency double-entry Journal Entries (JEs) and a full **Deferred Cost Rollforward Report** ready for NetSuite, SAP, or Workday Financials.

2. Functional Requirements & Accounting Logic

2.1 Cost Classification & Capitalization Rules

When a `Payout_Period` is locked in the Phase 1 engine, the ASC 606 module ingests all finalized `Commission_Ledger_Entry` records and evaluates them against configurable **Accounting Policy Rules**:

Cost Type	Default Treatment	Configurable Policy Toggles
Direct AE Commission (New Logo / Upsell)	**Capitalize**	Amortize over *Estimated Customer Life (ECL)* (e.g., 36–60 months) or *Contract Term*.
Renewal Commissions	**Capitalize (Contract Term)**	Test if renewal commission is *commensurate* with initial commission. If non-commensurate (typical), amortize initial commission over ECL and renewal commission strictly over the **Renewal Contract Term**.
Manager / Director Overrides	**Capitalize**	Legal & audit precedent treats direct hierarchy overrides on closed deals as incremental. Toggleable per role if tied to aggregate company EBITDA rather than contract closing.
Sales Engineering (SE) / Pooled Spiffs	**Configurable**	Capitalize if strictly formulaic per deal; Expense immediately if discretionary pool.
Fringe Benefits & Payroll Taxes	**Capitalize (Multiplier)**	Apply a configurable **Fringe Rate** (e.g., 8.5% for FICA/Medicare/401k match) paid *solely* as a result of the commission payout.
Practical Expedient (≤ 12 Months)	**Expense Immediately**	Under ASC 340-40-25-4, if the amortization period of the asset is 1 year or less, automatically bypass capitalization and expense in the period incurred.

2.2 Amortization Calculation Methods

The engine must support three configurable day-count and schedule conventions per Accounting Book:

1. **Exact Daily Straight-Line (365/366 Day Basis):**

$$\text{Monthly Amortization}_m = \text{Capitalized Asset} \times \left(\frac{\text{Days Active in Month } m}{\text{Total Days in Amortization Period}} \right)$$

2. **Full-Month Straight-Line (Mid-Month or Full-Month Convention):**

$$\text{Monthly Amortization}_m = \frac{\text{Capitalized Asset}}{\text{Total Months in Amortization Period}}$$

   Note: Any sub-cent rounding residual ($\pm \$0.01$ to $\$0.05$) is automatically absorbed in the final schedule period so that $\sum \text{Amortization} = \text{Initial Capitalized Asset}$ *exactly.*

3. **Performance Obligation (Milestones / Usage-Weighted):** Amortizes proportional to revenue recognition events ingested from the ERP when contract delivery is non-linear.

2.3 Contract Lifecycle Modifications & Edge Cases

Enterprise SaaS contracts rarely stay static for 3–5 years. The amortization engine must process four critical post-capitalization events without mutating closed historical accounting periods:

A. Full Contract Churn / Cancellation (Impairment)

- **Trigger:** CRM/ERP flags `Contract_Status = Churned` with an `Effective_Termination_Date`.
- **Action:**
  1. If within the commission clawback window, the Phase 1 engine generates a negative commission ledger entry, which reduces the Deferred Asset balance.
  2. Any remaining unamortized asset balance as of the `Effective_Termination_Date` is immediately written off via an **Impairment Loss Journal Entry** in the current open accounting period, and all future scheduled amortization rows are zeroed out.

B. Partial Downgrade / Contraction

- **Trigger:** Contract ARR decreases mid-term (e.g., customer drops 30% of seats).
- **Action:** Option (configurable by Controller) to either continue amortizing over remaining life (if expected future cash flows still exceed carrying value of the asset) or recognize a **pro-rata partial impairment** equal to the contraction percentage.

C. Retroactive Commission True-Ups (Late Deal Ingestion / Dispute Approval)

- **Trigger:** A deal closed in **January (Q1)**, but a commission dispute or late CRM booking adjustment is approved in **April (Q2)** after Q1 books are closed.
- **Action:** The system creates a **Child Amortization Stream** linked to the original January contract start date, and posts a **Cumulative Catch-Up Amortization Entry** in April for the elapsed Jan–Mar periods:

$$\text{April Catch-Up Expense} = \Delta\text{Asset} \times \left(\frac{\text{Elapsed Days (Jan 1 – Mar 31)}}{\text{Total Amortization Days}} \right)$$

D. Multi-Book Accounting (ASC 606 vs. IFRS 15 vs. Tax/Local GAAP)

- **Requirement:** Global enterprises must maintain parallel subledgers (e.g., US GAAP Parent vs. Statutory Local GAAP where commissions are expensed immediately).
- **Action:** Support multiple `Accounting_Book_ID` configurations running concurrently over the same `Commission_Ledger_Entry` dataset.

3. Double-Entry Journal Logic & Multi-Currency (ASC 830)

3.1 Standard Journal Entry Templates

Every accounting period close generates balanced Debit (`DR`) and Credit (`CR`) entries mapped to the customer's ERP Chart of Accounts (GL Codes):

Event Phase	GL Account Type	Account Name (Default Template)	Debit (DR)	Credit (CR)
1. Payroll Accrual	Expense (P&L)	Sales Commission Expense	$10,000	
(Commission Earned)	Liability (B/S)	Accrued Commissions Payable		$10,000
2. Capitalization	Asset (B/S - Short/Long Term)	Deferred Contract Acquisition Costs	$10,000	
(Deferral Offset)	Contra-Expense (P&L)	Commission Capitalization Offset		$10,000
3. Periodic Amortization	Expense (P&L)	Commission Amortization Expense	$277.78	
(Monthly Recognition)	Contra-Asset (B/S)	Accumulated Commission Amortization		$277.78
4. Churn Impairment	Expense (P&L)	Contract Asset Impairment Expense	$6,666.67	
(Unamortized Write-Off)	Contra-Asset (B/S)	Accumulated Commission Amortization		$6,666.67

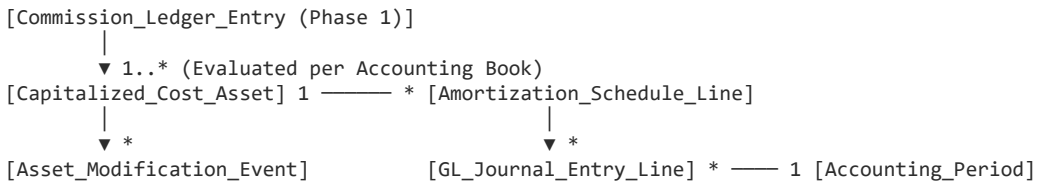
3.2 Short-Term vs. Long-Term Balance Sheet Reclassification

At the end of every reporting month, the system must automatically calculate the portion of the unamortized asset scheduled to amortize within the **next 12 months** (Current Asset) versus **beyond 12 months** (Non-Current Asset) and generate the reclassification journal entry.

3.3 Foreign Currency (FX) Handling (ASC 830 / IAS 21)

- **Non-Monetary Asset Rule:** Deferred contract acquisition costs are **non-monetary assets**.
- Therefore, the capitalized asset and its entire future amortization schedule MUST be locked at the **Historical Spot FX Rate** on the date the commission was capitalized into the entity's Functional Currency. It is **never revalued** at month-end closing spot rates (preventing unrealized FX gain/loss distortion on the amortization schedule).

4. Technical Data Schema (Phase 2 Extension)



Core Table Definitions

1. `Accounting_Book_Policy`

- `book_id` (UUID, PK)
- `book_name` (VARCHAR – e.g., `US_GAAP_ASC606`, `IFRS_15`, `STATUTORY_CASH`)
- `functional_currency` (CHAR(3))
- `amortization_method` (ENUM: `DAILY_EXACT`, `EQUAL_MONTHLY`, `FULL_MONTH_MID`)
- `practical_expedient_months` (INT, Default `12`)
- `default_customer_life_months` (INT, Default `36`)
- `fringe_benefit_rate` (NUMERIC(6,4), Default `0.0000`)

2. `Capitalized_Cost_Asset`

- `asset_id` (UUID, PK)
- `book_id` (UUID, FK)

- `source_ledger_id` (UUID, FK -> `Commission_Ledger_Entry.ledger_id`)
- `contract_id` (VARCHAR, Indexed – links to CRM/ERP Contract)
- `payee_id` (UUID, FK)
- `capitalization_date` (DATE, NOT NULL)
- `amortization_start_date` (DATE, NOT NULL)
- `amortization_end_date` (DATE, NOT NULL)
- `original_commission_amount_txn` (NUMERIC(18,4) – Payout currency)
- `historical_fx_rate` (NUMERIC(12,6) – Locked at `capitalization_date`)
- `capitalized_commission_func` (NUMERIC(18,4) – Functional currency)
- `capitalized_fringe_func` (NUMERIC(18,4))
- `total_initial_asset_func` (NUMERIC(18,4))
- `unamortized_balance_func` (NUMERIC(18,4))
- `asset_status` (ENUM: `ACTIVE`, `FULLY_AMORTIZED`, `IMPAIRED_CHURNED`, `EXPENSED_EXPEDIENT`)

3. `Amortization_Schedule_Line`

- `schedule_line_id` (UUID, PK)
- `asset_id` (UUID, FK, Indexed)
- `accounting_period_id` (UUID, FK – e.g., `2026-10`)
- `period_start_date` (DATE), `period_end_date` (DATE)
- `scheduled_amortization_func` (NUMERIC(18,4))
- `catchup_adjustment_func` (NUMERIC(18,4), Default `0.0000`)
- `impairment_amount_func` (NUMERIC(18,4), Default `0.0000`)
- `ending_short_term_balance_func` (NUMERIC(18,4) – Next 12 mos)
- `ending_long_term_balance_func` (NUMERIC(18,4) – Beyond 12 mos)
- `is_posted_to_gl` (BOOLEAN, Default `FALSE`)

4. `GL_Journal_Batch` & `GL_Journal_Entry_Line`

- `je_line_id` (UUID, PK)
- `batch_id` (UUID, FK)
- `accounting_period_id` (UUID, FK)
- `gl_account_number` (VARCHAR, NOT NULL)
- `department_cost_center` (VARCHAR)
- `subsidiary_entity_id` (VARCHAR)
- `debit_amount_func` (NUMERIC(18,4))
- `credit_amount_func` (NUMERIC(18,4))
- `source_schedule_line_id` (UUID, FK – provides 1-click drill-down from GL line to exact rep & deal)
- `erp_sync_status` (ENUM: `PENDING`, `EXPORTED`, `SYNCED_NETSUITE`, `FAILED`)

5. User Stories & Acceptance Criteria (Accounting Persona)

US 5.1: Automated Deferred Cost Rollforward Report (Audit Requirement)

- **Story:** *As a Revenue Accountant, I want to generate a monthly Deferred Commission Rollforward Schedule by cohort, contract, or payee so that I can reconcile the subledger to the General Ledger balance sheet during quarterly external audits.*
- **Acceptance Criteria:**
  - Report strictly satisfies the accounting identity for every row and period:

$$\text{Beginning Balance} + \text{Additions (Capitalized)} + \text{True-Ups} - \text{Amortization Expense} - \text{Impairments} = \text{Ending Bal}$$

*Ending Balance automatically splits into **Current (Short-Term)** and **Non-Current (Long-Term)** columns.

- Exportable to Excel/CSV with complete drill-down to underlying `transaction_id`, `payee_id`, and `source_ledger_id` in under 10 seconds for up to 500,000 active schedule lines.

US 5.2: Accounting Period Close & GL Push

- **Story:** *As a Controller, I want to preview the monthly amortization and capitalization journal entries, lock the accounting subledger period, and push the summarized JEs to NetSuite/SAP.*
- **Acceptance Criteria:**
  - System blocks Accounting Period Lock if total Debits $\neq$ total Credits across any subsidiary or currency.
  - Once `Accounting_Period` is locked, any subsequent commission adjustments or contract churns with an effective date in the closed period automatically roll forward into the next open accounting period as a `catchup_adjustment_func` or `impairment_amount_func`.

6. Audit & Compliance Verification Checklist

To pass Big-4 external audit scrutiny (Deloitte, PwC, EY, KPMG) for SOC 1 Type II and financial statement sign-off, the module enforces:

1. **Zero Orphaned GL Lines:** Every dollar in a summarized ERP Journal Entry can be traced back via foreign key (`source_schedule_line_id` $\rightarrow$ `asset_id` $\rightarrow$ `source_ledger_id` $\rightarrow$ `transaction_id`) to the original CRM contract and rep commission statement.
2. **Immutable Historical Schedules:** Modifying an accounting policy (e.g., extending Estimated Customer Life from 36 to 48 months) applies **prospectively** or generates an explicit adjustment event—never overwriting previously posted `is_posted_to_gl = TRUE` schedule lines.
3. **Automated Completeness Check:** A pre-close reconciliation gate verifies that 100% of locked `Commission_Ledger_Entry` payouts for the period have been evaluated by the ASC 606 classifier (either capitalized into `Capitalized_Cost_Asset` or explicitly tagged as `EXPENSED_EXPEDIENT`).

User prompt: Design the DAG calculation engine architecture

Response: Technical Architecture Specification: EIM Directed Acyclic Graph (DAG) Calculation Engine

Document Metadata	Details
Component Name	Core Deterministic DAG Calculation & State Engine
Performance SLA	500 Payees $\times$ 100,000 Transactions computed in < 15 seconds
Precision Standard	IEEE 754-2008 128-bit Decimal (`DECIMAL128` / `BigDecimal`), Zero Floating-Point Drift
Execution Paradigm	Functional, Immutable State, Topological Level-Synchronous Parallelism

1. Architectural Overview & Design Principles

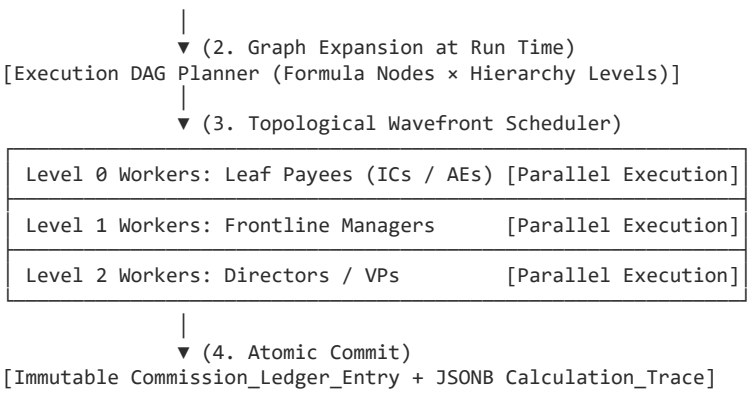
Enterprise commission calculation is fundamentally a **stateful, order-dependent graph problem**. Unlike standard OLAP queries that aggregate rows independently, commission rules require sequential state mutation (e.g., Deal 1 consumes the remaining $10,000 of Tier 1 quota, forcing Deal 2 into Tier 2, which then triggers a Manager Rollup override).

To achieve high throughput alongside cent-for-cent auditability, the engine is architected around four core principles:

1. **Two-Dimensional Dependency Resolution:** The engine resolves dependencies across both **Formula Logic** (Metric A $\rightarrow$ Quota Attainment $\rightarrow$ Rate Table Lookup $\rightarrow$ Payout) and **Organizational Hierarchy** (Leaf Rep $\rightarrow$ Frontline Manager $\rightarrow$ VP Rollup).
2. **Deterministic Total Ordering:** Because marginal tier splits depend on the sequence in which deals are processed, transactions within a period are sorted by a strict, deterministic tie-breaking composite key before graph evaluation.
3. **Zero-Mutation Retroactive True-Ups (Bi-Temporal Delta Engine):** Closed periods are never mutated. Retroactive data changes replay the historical graph in an isolated sandbox and emit an offsetting delta entry into the current open period.
4. **Full Lineage AST Tracing:** Every evaluated node emits both its scalar/vector result and a JSON Abstract Syntax Tree (AST) evaluation trace to power the Seller Statement "Calculation Trace Drawer" with zero runtime re-computation.

2. End-to-End Engine Pipeline

```
[No-Code Workbook / Plan Formulas]
                │
                ▼ (1. Parse & Compile at Plan-Save Time)
   [AST Parser & Type Checker] ──► (Cycle Detection: Tarjan's Algorithm)
```



3. Graph Structure: Nodes, Edges & Topological Wavefronts

3.1 Node Taxonomy

Every compensation plan compiles into an immutable Directed Acyclic Graph $G = (V, E)$, where vertices V represent strongly-typed data or transformation primitives:

Node Type	Granularity	Description	Example
`SourceNode`	Transaction	Raw filtered stream of ingested transactions or credits assigned to a payee.	`Deals.filter(Stage == 'Closed Won')`
`ParameterNode`	Payee / Period	Effective-dated scalar constants bound to the payee's plan assignment.	`Payee.Quota`, `Payee.Target_Incentive`
`StatefulWindowNode`	Cumulative Stream	Order-dependent running total that tracks pre-deal and post-deal state per transaction.	`RunningSum(Credited_ARR, order_by=Close_Date)`
`RateTableNode`	Marginal / Cliff	Evaluates a `StatefulWindowNode` interval against a multi-tier schedule.	`LookupTier(Cumulative_Attainment, AE_Rate_Table)`
`AggregateRollupNode`	Cross-Payee	Pulls evaluated outputs from child payees in the effective-dated hierarchy.	`Sum(DirectReports.Credited_ARR)`
`LedgerOutputNode`	Ledger Grain	Terminal sink node that emits immutable `Commission_Ledger_Entry` records.	`EmitPayout(Component='Base_ARR_Comm')`

3.2 Cycle Detection & Validation (Compile-Time)

When a Comp Admin saves a plan in the No-Code Builder, the compiler constructs the dependency graph and executes **Tarjan's Strongly Connected Components (SCC) algorithm** in $O(|V| + |E|)$ time:

- If any SCC has $|V| > 1$ (or a self-loop), the compiler rejects the save and highlights the exact circular formula reference in the UI (e.g., `Bonus_A depends on Net_Payout -> Net_Payout depends on Bonus_A`).
- Simultaneously, the **Hierarchy Graph Validator** checks the `Payee` reporting tree across all effective dates to prevent organizational loops (e.g., Manager A reports to Manager B on May 15 while Manager B reports to Manager A).

3.3 Wavefront Parallel Execution (Runtime)

Because a Sales Manager's commission often depends on the attainment or payouts of their direct reports, the scheduler partitions the execution graph into **Topological Wavefronts (Levels)**:

- **Wavefront 0 (Leaf Nodes):** All Individual Contributors (AEs, SDRs, SEs) with zero downline dependencies. If an enterprise has 450 AEs, all 450 AE sub-graphs are completely independent and execute concurrently across worker threads.
- **Wavefront 1 (First-Line Managers):** Unlocks immediately once all child nodes in Wavefront 0 complete and materialize their outputs in the in-memory state store.
- **Wavefront N (Executives/VPs):** Executes sequentially up the hierarchy tree.

4. Core Algorithmic Deep-Dives

4.1 Deterministic Marginal Tier-Splitting Algorithm

The hardest mathematical requirement in EIM is **Marginal (Progressive) Tier Splitting**—where a single deal pushes a rep across one or more quota hurdles and must be sliced across multiple commission rates.

Step 1: Deterministic Transaction Ordering

Before evaluating a `StatefulWindowNode`, all credited transactions for Payee p in Period t are sorted by a strict composite key:

$$\text{SortKey} = (\text{Transaction_Date ASC}, \text{External_Timestamp_UTC ASC}, \text{Transaction_UUID ASC})$$

This guarantees that running the calculation today or 3 years from now during an audit produces the exact same deal slicing sequence.

Step 2: Interval Intersection Math

For each transaction i with credited amount $\Delta C_i > 0$:

1. Let A_{prev} be the cumulative attainment prior to transaction i, and $A_{\text{curr}} = A_{\text{prev}} + \Delta C_i$ be the new cumulative attainment.
2. Transaction i occupies the attainment interval $I_i = [A_{\text{prev}}, A_{\text{curr}}]$.
3. Given a Rate Table with K tiers defined by quota intervals $T_k = [L_k, U_k]$ and rates r_k (for $k = 1 \dots K$):
4. For each tier k, compute the overlapping slice $\Delta S_{i,k}$:

$$\Delta S_{i,k} = \max\left(0, \min(A_{\text{curr}}, U_k) - \max(A_{\text{prev}}, L_k)\right)$$

5. If $\Delta S_{i,k} > 0$, emit a **Marginal Slice Sub-Entry**:

$$\text{Payout}_{i,k} = \Delta S_{i,k} \times r_k$$

Concrete Execution Example

- **Setup:** Quota = $\$100,000$. Tier 1 ($0$–$100\%$): 10%. Tier 2 (100–150%): 15%. Tier 3 ($> 150\%$): 20%.
- **State:** Rep is at $A_{\text{prev}} = \$90,000$ and closes a single $\$70,000$ deal ($A_{\text{curr}} = \$160,000$).
- **Engine Output (3 Atomic Ledger Slices for 1 Transaction):**
  - **Slice 1 (Tier 1):** $\min(160\text{k}, 100\text{k}) - \max(90\text{k}, 0) = \$10,000 \times 10\% = \$1,000.00$
  - **Slice 2 (Tier 2):** $\min(160\text{k}, 150\text{k}) - \max(90\text{k}, 100\text{k}) = \$50,000 \times 15\% = \$7,500.00$
  - **Slice 3 (Tier 3):** $\min(160\text{k}, \infty) - \max(90\text{k}, 150\text{k}) = \$10,000 \times 20\% = \$2,000.00$
  - **Total Deal Payout:** $\$10,500.00$ (with all 3 slices persisted for the Rep Calculation Trace).

4.2 Mid-Period Effective-Dated Proration & Hierarchy Splits

Enterprise reps frequently change quotas, plans, or managers mid-month (e.g., promoted from Mid-Market AE to Enterprise AE on June 15).

- **Segment Partitioning:** When `Plan_Assignment` or `Payee` contains multiple effective-dated records overlapping a single `Payout_Period` $[D_{\text{start}}, D_{\text{end}}]$, the DAG planner splits the Payee's execution context into **Sub-Period Segments** $S_1, S_2, \dots, S_m$.
- **Quota Proration Formula:**

$$\text{Prorated_Quota}_{S_j} = \text{Full_Period_Quota}_{S_j} \times \left(\frac{\text{Active Calendar Days in } S_j}{\text{Total Calendar Days in Period}} \right)$$

- Transactions are routed to segment S_j based on `transaction_date` $\in [S_{j,\text{start}}, S_{j,\text{end}}]$, ensuring deals closed before June 15 evaluate against the Mid-Market rate table and roll up to the Mid-Market Manager, while deals on or after June 15 evaluate against the Enterprise rate table.

4.3 Bi-Temporal Retroactive True-Up Engine (Shadow Replay)

When a historical period P_{closed} (e.g., March) is `LOCKED`, its `Commission_Ledger_Entry` rows are immutable. However, in May (P_{open}), Finance may discover a March deal was under-booked by $\$20,000$ in Salesforce, which also would have pushed the rep into an accelerator for *all subsequent March deals*.

Simply calculating the $\$20,000$ difference in isolation is mathematically incorrect because of the ripple effect on subsequent marginal tiers. The engine solves this via **Bi-Temporal Shadow Replay**:

1. **Snapshot Load:** Load the frozen input state of P_{closed} (as of the March lock timestamp T_{lock}) and merge the newly discovered retroactive transaction modifications as of current timestamp T_{now}.
2. **Shadow DAG Execution:** Re-run the exact versioned DAG of P_{closed} in an isolated memory space to produce the **Shadow Ledger** $L_{\text{shadow}}(P_{\text{closed}})$.
3. **Set-Based Diffing:** Compare the Shadow Ledger against the Immutable Historical Ledger $L_{\text{locked}}(P_{\text{closed}})$ across every `(payee_id, transaction_id, plan_component_id)` tuple:

$$\Delta\text{Payout} = L_{\text{shadow}}.\text{payout_amount} - L_{\text{locked}}.\text{payout_amount}$$

4. **True-Up Emission:** For every tuple where $\Delta\text{Payout} \neq 0$, write a new ledger record into the **current open period** ($P_{\text{open}} = \text{May}$) with:

- `entry_type = 'PRIOR_PERIOD_TRUEUP'`
- `origin_period_id = '2026-03'`
- `payout_amount =` ΔPayout

4.4 Recoverable Draw State Machine

Draws act as a stateful balance sheet carry-forward node at the end of a Payee's DAG:

```
Let Earned_Comm = Sum(Calculated_Commissions_In_Period)
Let Guaranteed_Draw = Parameter(Payee.Monthly_Draw_Amount)
Let Prior_Deficit = StateStore.get(Payee, Period - 1).Ending_Draw_Deficit

If (Earned_Comm < Guaranteed_Draw) AND (Is_Draw_Active == TRUE):
    Advance_Payout = Guaranteed_Draw - Earned_Comm
    Net_Payout = Guaranteed_Draw
    Ending_Draw_Deficit = Prior_Deficit + Advance_Payout  // If Recoverable
Else If (Earned_Comm >= Guaranteed_Draw):
    Max_Recoverable = Earned_Comm - Guaranteed_Draw      // Or 100% of Earned_Comm per plan rule
    Draw_Recovery = Min(Prior_Deficit, Max_Recoverable)
    Net_Payout = Earned_Comm - Draw_Recovery
    Ending_Draw_Deficit = Prior_Deficit - Draw_Recovery
```

5. Calculation Trace Payload & Audit Lineage

To eliminate seller disputes and power the **Calculation Trace Drawer** (US 4.1 in the PRD), every `Commission_Ledger_Entry` stores a compressed JSONB lineage tree (`calculation_trace`) generated during DAG traversal:


```
{
  "ledger_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "plan_version_hash": "sha256:8f434346648f6b96df89dda901c5176b10a6d83961dd3c1ac88b59b2dc327aa4",
  "transaction_context": {
    "external_deal_id": "OPP-2026-88412",
    "deal_name": "Acme Corp Enterprise Expansion",
    "raw_booking_amount": "100000.0000",
    "currency": "USD",
    "split_percentage": "0.7000",
    "credited_amount": "70000.0000"
  },
  "attainment_window_state": {
    "quota_target": "100000.0000",
    "prior_cumulative_attainment": "90000.0000",
    "prior_attainment_pct": "0.900000",
    "post_cumulative_attainment": "160000.0000",
    "post_attainment_pct": "1.600000"
  },
  "marginal_tier_slices": [
    {
      "tier_name": "Tier 1 (0% - 100% Quota)",
      "slice_amount": "10000.0000",
      "rate_applied": "0.100000",
      "slice_payout": "1000.0000"
    },
    {
      "tier_name": "Tier 2 Accelerator (100% - 150% Quota)",
      "slice_amount": "50000.0000",
      "rate_applied": "0.150000",
      "slice_payout": "7500.0000"
    },
    {
      "tier_name": "Tier 3 Super-Accelerator (>150% Quota)",
      "slice_amount": "10000.0000",
      "rate_applied": "0.200000",
      "slice_payout": "2000.0000"
    }
  ],
  "final_rounded_payout": "10500.00",
  "rounding_mode": "HALF_EVEN"
}
```


6. Systems Stack, Scaling & Fault Tolerance

Architectural Layer	Recommended Technology	Rationale
Core Calculation Worker	**Rust** (or **Go** with `shopspring/decimal`)	Memory safety, zero garbage-collection pauses during massive array allocations, native 128-bit fixed-point decimal support (`rust_decimal`), and fearless concurrency via Rayon thread pools.
In-Memory Vector Format	**Apache Arrow**	Columnar in-memory layout allows zero-copy data transfer between Snowflake/PostgreSQL ingestion and the Rust DAG worker nodes.
Workflow Orchestrator	**Temporal.io**	Manages distributed wavefront state, automatic retries on worker failure, and long-running batch orchestration with guaranteed exactly-once execution semantics.
Transactional Ledger Store	**PostgreSQL 16+** (Partitioned by `payout_period_id`)	ACID transactions ensure that a calculation run for a payout period either commits 100% of ledger entries atomically or rolls back cleanly.
Incremental Dirty-Flag Cache	**Redis Cluster**	Hashes `(Payee_SubGraph_AST + Input_Transactions_Hash)`. During interactive admin plan modeling, if a change only affects Enterprise AEs, the engine skips recalculating SDRs and Mid-Market AEs, returning preview results in $< 800\text{ms}$.

User prompt: Write detailed UX/UI wireframe specifications and user flows for the No-Code Plan Builder and the Seller Statement Calculation Trace drawer.

Response: UX/UI Wireframe Specifications & User Flows: No-Code Plan Builder & Calculation Trace Drawer

Document Metadata	Details
Target Modules	1. Admin No-Code Plan Builder (RevOps Persona) 2. Seller Statement & Calculation Trace Drawer (Sales Rep Persona)
Design System	High-Density Enterprise Data UI (8px Spatial Grid, Tabular Monospace Numbers `font-variant-numeric: tabular-nums`)
Core UX Goal	Eliminate the "Black Box" of legacy EIM tools by making complex DAG logic visually self-documenting for both Admins and Sellers.

Part 1: Admin No-Code Plan Builder (RevOps Persona)

1.1 UX Philosophy: "Structured Workbook + Live DAG Preview"

Legacy ICM platforms force admins to navigate 15 nested modal forms to create a single rate table, while pure spreadsheets lack type safety and break on effective-dated changes. Our **Hybrid Split-Canvas UX** combines:

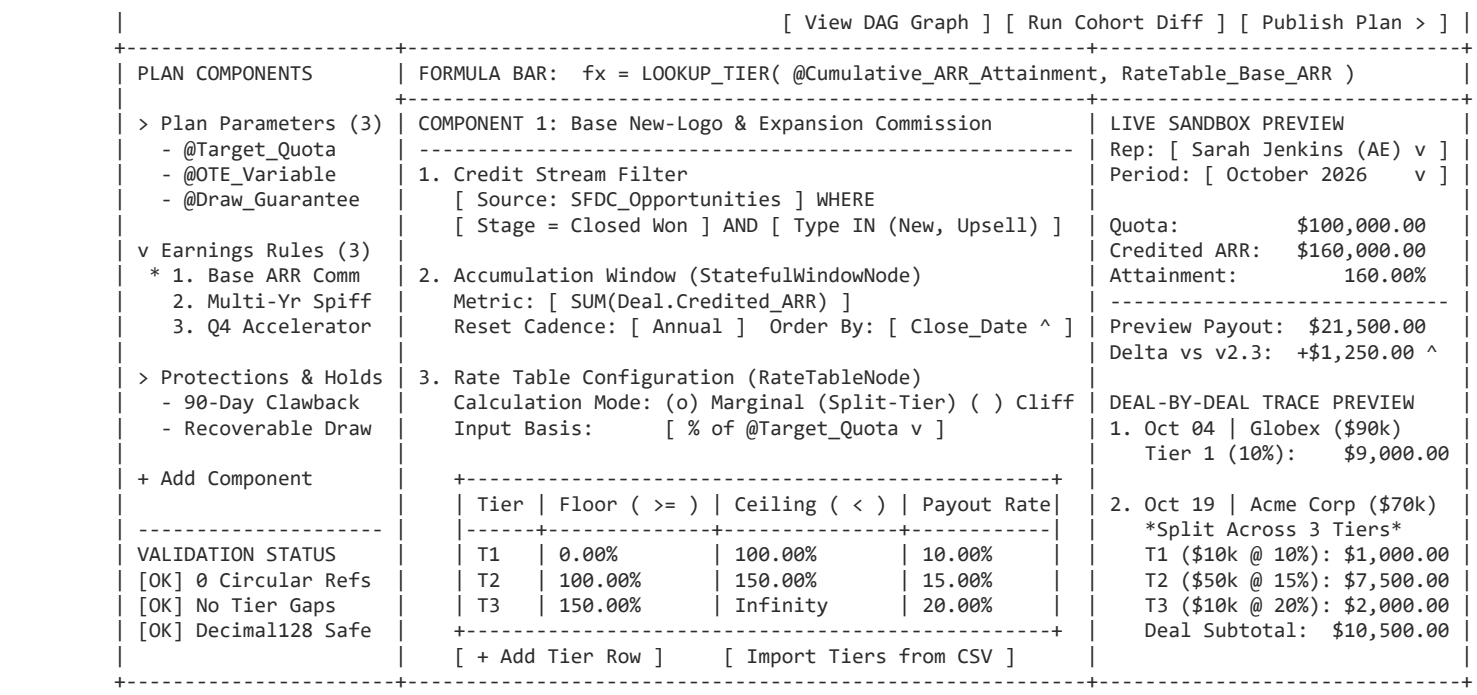
- **Left/Center Canvas:** A strongly-typed, block-based **Formula & Rate Table Workbook** (familiar to Excel/CaptivateIQ users).
- **Right Dock:** A **Live Payee Preview & DAG Dependency Inspector** that recalculates a sample rep's payout in real time ($< 800\text{ms}$ via Redis dirty-flag cache) as the admin edits rules.

1.2 End-to-End Admin User Flow

1. **Plan Initialization:** Admin creates or clones a plan (`FY26 Enterprise AE Plan`), sets effective dates (`2026-01-01` to `2026-12-31`), and assigns target roles/cohorts.
2. **Data Source & Credit Binding:** Admin selects which ingested transaction stream feeds the plan component (e.g., `Salesforce Closed-Won New Logo & Upsell`) and defines split/rollup rules.
3. **Component & Rate Table Authoring:** Admin adds calculation blocks (e.g., `Cumulative Quota Attainment` $\rightarrow$ `Marginal Rate Table` $\rightarrow$ `Multi-Year Spiff`) using autocomplete tokens and toggles between **Marginal** and **Cliff** modes.
4. **Live Sandbox Verification:** Admin selects a real rep (`Sarah Jenkins - Enterprise AE`) and a period (`Q3 2026`) in the Right Dock to inspect immediate deal-by-deal payout outputs and verify edge cases (such as split-tier crossings).
5. **Compile & Publish Gate:** Admin clicks **Validate & Publish**. The UI runs Tarjan's cycle check and schema validation, displays a diff of total payout impact across all assigned reps, and commits the new `plan_version_hash`.

1.3 Wireframe Specification: No-Code Plan Builder Canvas

```
+-----------------------------------------------------------------------------------------------------------+
| [ < Back to Plans ]  FY26 Enterprise AE Compensation Plan   [v2.4 DRAFT]   Effective: Jan 1, 2026 - Dec 31, 2026 |
```



1.4 Component-Level UI Specifications & Micro-Interactions (Plan Builder)

UI Element	Interaction & Behavioral Specification	Error & Edge-Case Handling
Tokenized Formula Bar (`fx`)	Typing `@` triggers an autocomplete popover of **Plan Parameters & Upstream Nodes** (`@Target_Quota`, `@Prior_Attainment`). Typing `$` triggers **Transaction Attributes** (`$Deal.Booking_Amount`, `$Deal.Term_Months`). Color-codes tokens by data type (`Currency` = Emerald, `Percentage` = Indigo, `Boolean` = Amber).	Real-time AST type-checker underlines type mismatches in red (e.g., multiplying a `Date` by a `Currency`) with an inline fix suggestion before the user leaves the field.
Rate Table Grid (`RateTableNode`)	Clicking **Marginal (Split-Tier)** vs. **Cliff (Retroactive)** dynamically renders a visual mini-diagram explaining how a threshold-crossing dollar is treated. Ceiling values in Row N automatically bind and auto-populate the Floor value of Row $N+1$ (`100.00%` $\rightarrow$ `100.00%`).	**Contiguity Enforcement:** It is impossible to create a gap (e.g., Tier 1 ends at `100%` and Tier 2 starts at `101%`) or an overlap. Deleting a middle row automatically heals the adjacent interval boundary.
Live Sandbox Preview Dock	Features a searchable Payee selector and Period selector. Also allows the admin to inject a **"Synthetic Test Deal"** (e.g., *"Add hypothetical $50k deal on Oct 28"*) to test edge cases without touching real CRM data.	If a formula change increases total cohort payout by more than a configurable threshold (e.g., $> 10\%$), an amber warning badge (`High Financial Variance Detected`) prompts the admin to run a full Cohort Diff before publishing.

Part 2: Seller Statement & Calculation Trace Drawer (Sales Rep Persona)

2.1 UX Philosophy: "Self-Service Auditability to Eliminate Shadow Accounting"

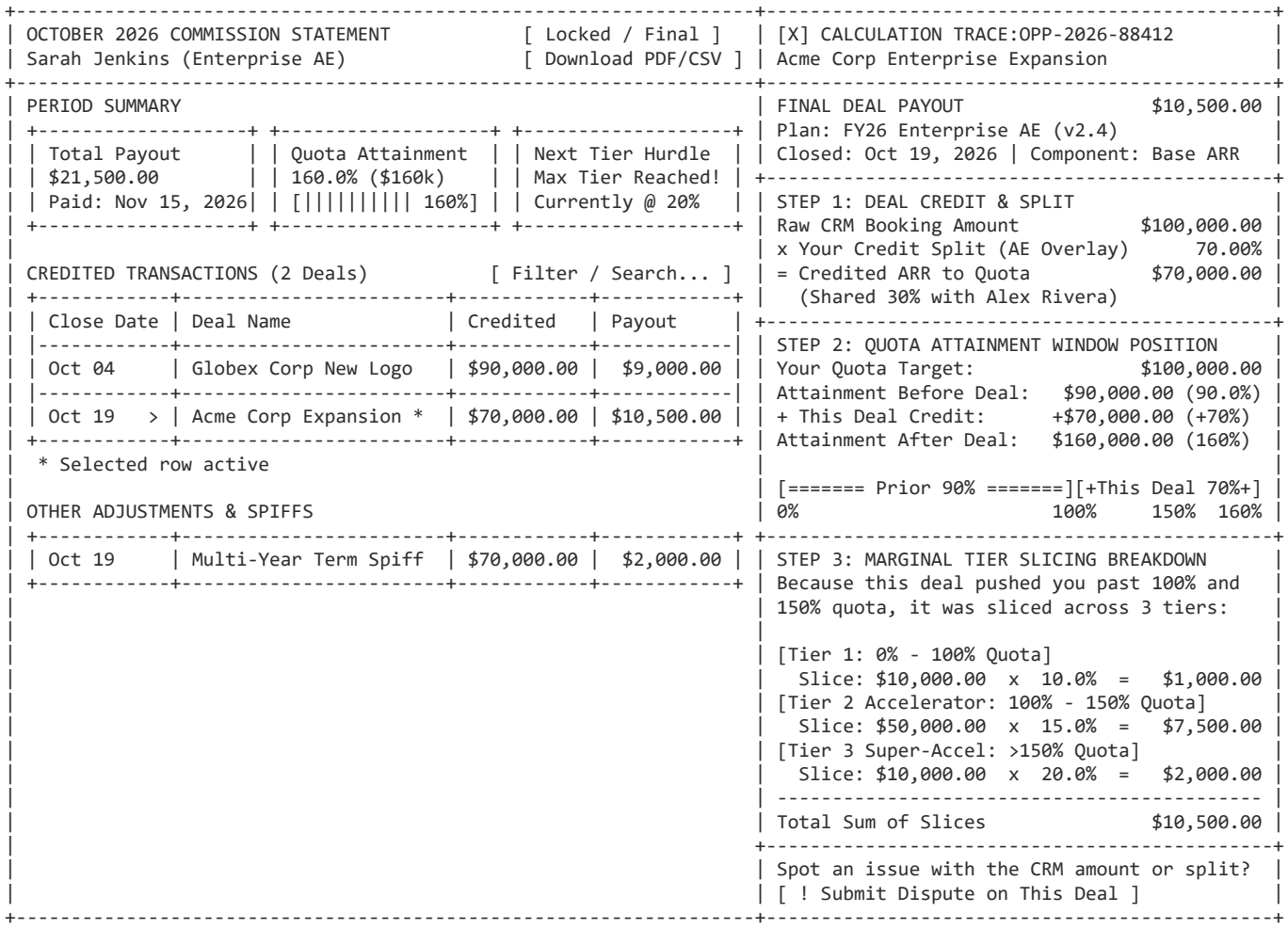
Sales reps create personal shadow spreadsheets because legacy statements only show a final row: `Acme Corp | $100,000 | $10,500 Commission`. When a deal crosses multiple marginal tiers or includes a split, the effective rate (`10.5%`) looks wrong to the rep, triggering a dispute ticket.

Our **Calculation Trace Drawer** directly renders the `calculation_trace` JSONB payload from the DAG engine as a visual **Domino Waterfall**—showing the exact math from raw CRM booking $\rightarrow$ split credit $\rightarrow$ quota window position $\rightarrow$ tier slices $\rightarrow$ final payout.

2.2 End-to-End Seller User Flow

1. **Statement Overview:** Seller logs into the EIM portal (or embedded CRM tab) and views their current period summary: **Total Payout**, **Quota Attainment Thermometer**, and **Component Breakdown** (Base Commissions, Spiffs, Draws/True-Ups).
2. **Deal Inspection:** Seller notices `Acme Corp Enterprise Expansion ($100,000)` paid `$10,500.00`—which doesn't match a simple 10% or 15% flat multiplication—and clicks the row.
3. **Trace Drawer Slide-Over:** A 560px right-hand drawer slides over the statement, visually breaking down the **70% Deal Split** and the **3-Tier Marginal Accelerator Slicing**.
4. **In-Context Dispute (Exception Path):** If the rep notices the split should have been 100% instead of 70%, they click **"Flag Issue on This Step"** directly on Step 1 of the trace drawer, pre-populating the exact `ledger_id`, `transaction_id`, and disputed node for RevOps.

2.3 Wireframe Specification: Seller Statement & Calculation Trace Drawer



2.4 Component-Level UI Specifications & Micro-Interactions (Seller Trace Drawer)

UI Component	Visual & Interactive Behavior	Data Binding (`calculation_trace` JSONB)
Stacked Attainment Bar (Step 2)	A horizontal segmented progress bar with vertical tick marks at every quota tier boundary (`100%`, `150%`). Displays `Prior Attainment` in muted slate grey and animates the `Current Deal` segment across the tier boundaries in vibrant blue/emerald gradients.	Bound to `attainment_window_state.prior_attainment_pct`, `post_attainment_pct`, and rate table thresholds.
Marginal Slice Cards (Step 3)	Renders one card per non-zero slice in `marginal_tier_slices[]`. Hovering over a slice card highlights the corresponding segment on the Stacked Attainment Bar in Step 2 so the seller immediately sees *which portion* of the deal earned `10%`, `15%`, and `20%`.	Bound directly to `marginal_tier_slices[].slice_amount`, `rate_applied`, and `slice_payout`.
Contextual Dispute Modal Trigger	Clicking `[ ! Submit Dispute on This Deal ]` opens an inline step selector: *"What looks incorrect?"* with 4 structured radio options: 1. `CRM Deal Amount ($100,000)` 2. `Credit Split % (70%)` 3. `Prior Attainment Balance ($90,000)` 4. `Rate / Plan Rule Applied`	Pre-tags the `Dispute_Ticket` with the exact DAG node category (`SOURCE_DATA`, `CREDIT_SPLIT`, `WINDOW_STATE`, or `PLAN_LOGIC`), allowing RevOps to triage and route disputes 3x faster.

UI Component	Visual & Interactive Behavior	Data Binding (`calculation_trace` JSONB)
Prior-Period True-Up Badge	If `entry_type == 'PRIOR_PERIOD_TRUEUP'`, the drawer header renders a high-contrast purple badge: `Retroactive Adjustment from March 2026` and shows a side-by-side diff table (`Original March Payout: $4,200` vs. `Recalculated March Payout: $5,000` → `Net October Adjustment: +$800`).	Bound to `origin_period_id` and the Shadow Replay diff payload.

3. Accessibility, Responsiveness & Design Tokens

1. **Tabular Numeric Alignment:** Every currency, percentage, and attainment figure across both the Admin Grid and Seller Statement enforces `font-variant-numeric: tabular-nums` (using `JetBrains Mono` or `Inter Tabular`) so decimal points align vertically down every column.
2. **Colorblind-Safe Semantic Palette (WCAG 2.1 AA):**
   - **Base Tier / Normal State:** Slate Blue (`#2563EB`)
   - **Accelerated Tier / Positive Delta:** Emerald (`#059669`) + Upward Chevron icon `^`
   - **Clawback / Draw Recovery / Negative True-Up:** Rose (`#E11D48`) + Explicit minus sign `-` (never rely on red text alone)
   - **Locked / Immutable Period:** Neutral Zinc (`#52525B`) with padlock icon
3. **Keyboard-First Admin Navigation:** The No-Code Plan Builder supports full spreadsheet keyboard shortcuts (`Tab`, `Shift+Tab`, `Enter` to commit cell, `Cmd/Ctrl + Z` for AST undo history, and `Cmd/Ctrl + K` to open the command palette and jump to any payee preview).
